python闭包装饰器与深浅拷贝

黑马程序员
www.itheima.com
传智教育旗下
高端IT教育品牌

目录

Contents

- 闭包

  概念和作用、构成条件、 nonlocal

- 装饰器

  概念和作用、构成条件、传统方式和语法糖方式

- 装饰器的使用

  有(无)参有(无)返回值的原函数装饰、不定长参数原函数装饰

- 多个装饰器的使用

  多个装饰器装饰一个原函数

- 带有参数装饰器

  一个装饰器装饰多个原函数

- 深浅拷贝

  浅拷贝、深拷贝

目录

Contents

- 闭包

  概念和作用、构成条件、 nonlocal
- 装饰器
- 装饰器的使用
- 多个装饰器的使用
- 带有参数装饰器
- 深浅拷贝

学习目标

Learning Objectives

1. 知道闭包的构成条件
2. 知道定义闭包的语法格式
3. 能够编写闭包的代码

闭包的作用

疑问

在之前的学习中，我们会发现：

当调用完函数后，函数内定义的**变量就销毁了**，但有时需要**保存函数内的这个变量**，并在这个变量的基础上完成一系列的操作

比如：每次在这个变量的基础上和其它数字进行求和计算，**那怎么办呢?**

闭包

闭包的作用

闭包可以保存函数内的变量，而不会随着调用完函数而被销毁。

案例分析

定义一个函数用于保存变量10，然后调用函数返回值变量并重复累加数值，观察效果。

```
# 1 定义一个函数 返回函数运行结果
# 2 想在函数内部累加 number+1+2, 但做不到

# 1 定义一个函数 返回函数运行结果
def myfun():
    number = 10
    return number

# 2 想在函数内部累加 number+1+2, 但做不到
number = myfun()
print(number)
print(number+1)
print(number+2)
print(number+3)
```

闭包的语法

在函数嵌套的前提下，内部函数使用了外部函数的变量，并且外部函数返回了内部函数，

这种：使用外部函数变量的内部函数称为闭包。

```
# 外部函数
def 外部函数名(外部参数):
    # 内部函数
    def 内部函数名(内部参数):
        ...[使用外部函数的变量]
    return 内部函数名        # 闭包
```

闭包的构成条件

闭包的构成条件

①**有嵌套**：在函数嵌套(函数里面再定义函数)的前提下；

②**有引用**：内部函数使用了外部函数的变量(还包括外部函数的参数)；

③**有返回**：外部函数返回了内部函数名。

三个条件

使用闭包

案例分析

定义一个用于求和的闭包。
其中，外部函数有参数num1，内部函数有参数num2，然后调用，并用于求解两数之和，观察效果。

```
# 闭包的构成条件:
# 1在函数嵌套(函数里面再定义函数)的前提下
def func_out(num1):
    def func_inner(num2):
# 2内部函数使用了外部函数的变量(还包括外部函数的参数)
        num = num1 + num2
        print("现在的值：", num)
# 3外部函数返回了内部函数
    return func_inner


# 创建闭包实例
f = func_out(10)
# 执行闭包
f(1)
```

思考：闭包的代码的执行流程是什么样的？

nonlocal关键字

global：声明全局变量
nonlocal：声明能够让内部函数去修改外部函数的变量

案例分析

编写一个闭包，让内部函数去访问外部函数内的参数a = 100，观察效果。

```
# 定义
def func_out():
    a = 100
    def func_inner():
        # 声明变量
        nonlocal a  # 注意要声明
        a = a + 1 # 修改值  有引用
        print(f"修改的结果:{a}")
    return func_inner

# 观察结果
number = func_out()
number()
```

总结

1.闭包的作用？

保存函数中的变量（外部函数中的变量）

2.闭包的语法格式?

```
# 外部函数
def 外部函数名(外部参数):
    # 内部函数
    def 内部函数名(内部参数):
        ...[使用外部函数的变量]
    return 内部函数名        # 闭包
```

3.构成闭包的条件?

有嵌套，有引用，有返回

4. nonlocal关键字的作用？

声明能够让内部函数去修改外部函数的变量

目录

Contents

- 闭包
- 装饰器

  作用、构成条件、传统方式和语法糖方式
- 装饰器的使用
- 多个装饰器的使用
- 带有参数装饰器
- 深浅拷贝

学习目标

Learning Objectives

1. 能够说出装饰器的作用
2. 知道装饰器的构成条件
3. 知道装饰器语法

装饰器

装饰器的作用是**不改变原有函数的基础上，给原有函数增加额外功能。**

装饰器本质上就是一个闭包函数。

装饰器的构成条件

①**有嵌套**：在函数嵌套(函数里面再定义函数)的前提下；

②**有引用**：内部函数使用了外部函数的变量(还包括外部函数的参数)；

③**有返回**：外部函数返回了内部函数名；

④**有额外功能**：给需要装饰的原有函数增加额外功能。

四个条件

装饰器语法

方式1：**传统方式**：　变量名 = 装饰器名(原有函数名)
　　变量名()

案例分析

例如，发表评论前，都是需要先登录的。
先定义有发表评论的功能函数，然后在**不改变原有函数的基础上**，需要提示用户要先登录。

```
# 1定义一个装饰器(装饰器的本质是闭包)
def check(fn):
    def inner():
        print("登陆验证。。。")
        fn()

    return inner

# 2.需要被装饰的函数
def comment():
    print("发表评论")

# 3.使用装饰器装饰函数（增加一个登陆功能）
comment = check(comment)
comment()
```

装饰器语法

方式2：**语法糖**：@装饰器名　（这种方式最常见也最常用）

案例分析

例如，发表评论前，都是需要先登录的。
先定义有发表评论的功能函数，然后在**不改变原有函数的基础上**，需要提示用户要先登录。

```
# 1定义一个装饰器(装饰器的本质是闭包)
def check(fn):
    def inner():
        print("请先登陆")
        fn()
    return inner

# 2使用装饰器装饰函数（增加一个登陆功能）
# 解释器遇到@check 会立即执行comment =
check(comment)
@check
def comment():
    print("发表评论")
comment()
```

总结

1. 能够说出装饰器的作用

   在不改变原有函数的基础上，给原有函数添加新功能

2. 知道装饰器的构成条件

   有嵌套，有引用，有返回，有额外功能

3. 知道装饰器语法糖的写法

方式1：**传统方式**：　变量名 = 装饰器名(原有函数名)
　　变量名()
方式2：**语法糖**：@装饰器名

目录

Contents

- 闭包
- 装饰器
- 装饰器的使用

  有(无)参有(无)返回值的原函数装饰、不定长参数原函数装饰
- 多个装饰器的使用
- 带有参数装饰器
- 深浅拷贝

装饰器的使用

函数分类

无参无返回值的函数

定义：def 函数名()：...
调用：函数名()

有参无返回值的函数

定义：def 函数名(形参)：...
调用：函数名(实参)

无参有返回值的函数

定义：def 函数名()：... return 返回值
调用：用变量接收返回值 = 函数名()

有参有返回值的函数

定义：def 函数名(形参)：... return 返回值
调用：用变量接收返回值 = 函数名(实参)

装饰器的使用

装饰器装饰无参无返回值的函数

需求

在无参无返回值的原有函数求和计算结果之前，添加一个友好提示(注意：不能改变源码)：正在努力计算中...。

```
#1. 定义装饰器
def print_info(func):
    def inner(): # 有嵌套
        print("【友好提示】正在努力计算中...") # 有额外功能
        func() # 有引用
    return inner # 有返回
# 2.定义函数,并添加语法糖
@print_info
def get_sum():
    a = 10
    b = 20
    c = a + b
    print(f"两数之和为:{c}")
# 3.调用
get_sum()
```

装饰器的使用

装饰器装饰有参无返回值的函数

需求

在**有参无返回值**的原有函数求和计算结果之前，添加一个友好提示(注意：不能改变源码)：正在努力计算中...。

```
# 1.定义装饰器
def print_info(func):
    def inner(a,b): # 有嵌套
        print("【友好提示2】努力计算中...") # 有额外功能
        func(a,b) # 有引用
    return inner # 有返回

# 2.定义函数,添加语法糖
@print_info
def get_sum(x,y):
    z = x + y
    print(f"两数之和为:{z}")

# 3.调用
get_sum(20,35)
```

装饰器的使用

装饰器装饰无参有返回值的函数

需求

在**无参有返回值**的原有函数求和计算结果之前，添加一个友好提示(注意：不能改变源码)：正在努力计算中...。

```
# 1.定义装饰器
def print_info(func):
    def inner():  # 有嵌套
        print("【友好提示】正在努力计算中...")
        result = func()   # 有引用
        return result  # 原有函数有返回值
    return inner  # 有返回
# 2.定义函数,添加语法糖
@print_info
def get_sum():
    x = 19
    y = 34
    z = x + y
    return z  # 求和
# 3.调用
print(get_sum())
```

装饰器的使用

装饰器装饰有参有返回值的函数

需求

在**有参有返回值**的原有函数求和计算结果之前，添加一个友好提示(注意：不能改变源码)：正在努力计算中...。

```
# 1.定义装饰器
def print_info(func):
    def inner(a,b):  # 有嵌套
        # 有额外功能
        print("【友好提示】正在努力计算中...")
        result = func(a,b)  # 引用
        return result
    return inner  # 有返回
# 2.定义函数,并添加语法糖
@print_info
def get_sum(x,y):
    z = x + y
    return z
# 3.调用
print(get_sum(13,45))
```

通用装饰器的使用（函数参数是不定长的，有返回值）

需求

定义一个可以计算多个数据和多个字典value值之和的函数，并调用。

在原有函数的计算结果之前，加一个友好提示(不能改变源码)：正在努力计算中...。

*args: 元组类型
**kwargs: 字典类型

```
# 1.定义装饰器
def print_info(func):
    def inner(*args,**kwargs):  # 有嵌套
        print("【友好提示】正在努力计算中...")
        return func(*args,**kwargs)  # 有引用
    return inner  # 有返回

# 2.定义函数，并添加语法糖
@print_info   # args元组，  kwargs字典
def get_sum(*args, **kwargs):
    result = 0
    for value in args:
        result += value
    for value in kwargs.values():
        result += value
    return result
# 3.调用
get_sum(1,3,4,5,a=10,b=20,c=30)
```

总结

1. 装饰器装饰4种不同分类的函数：
   - a 当原有函数没有参数时，对应定义的装饰器的内部函数也没有参数
   - b 当原有函数没有返回值时，对应定义的装饰器的内部函数也没有返回值
   - c 当原有函数有返回值时，定义的装饰器的内部函数中要返回值
   - d 当原有函数有参数时，定义的装饰器类的内部函数也应该有参数

总之一句话：原函数是有参无参，有返回值无返回值，装饰器的内部函数和原函数保持一致

目录

Contents

- 闭包
- 装饰器
- 装饰器的使用
- 多个装饰器的使用

  多个装饰器装饰一个原函数
- 带有参数装饰器
- 深浅拷贝

学习目标

Learning Objectives

能够使用多个装饰器装饰一个函数

多个装饰器装饰一个函数

案例分析

例如，发表评论前，都是需要 先登录用户 再进行 验证码验证。
先定义有发表评论的功能函数，然后在不改变原有函数的基础上，需要先检查用户登录和输入验证码。

多个装饰器的装饰过程是：离函数最近的装饰器先装饰，然后外面的装饰器再进行装饰，由内到外的装饰过程

```
#1.定义装饰器1
def check_user(fn):
    def inner():
        print("输入用户名和密码")
        fn()
    return inner
#2.定义装饰器2
def check_code(fn):
    def inner():
        print("登录验证码")
        fn()
    return inner
# 3. 被装饰器的函数
@check_user
@check_code
def comment():
    print("发表评论")
# 4.调用函数
comment()
```

目录

Contents

- 闭包介绍
- 闭包的使用
- 装饰器
- 装饰器的使用
- 多个装饰器的使用
- 带有参数装饰器

  一个装饰器装饰多个原函数
- 深浅拷贝

带有参数的装饰器

需求

定义一个**既能装饰减法运算，又能装饰加法运算**的装饰器，即带有参数的装饰器。

```
# 装饰器
# 外部函数
def decorator(fn, flag):
    # 内部函数
    def inner(num1, num2):
        # 判断流程
        if flag == "+":
            print("--正在努力加法计算--")
        elif flag == "-":
            print("--正在努力减法计算--")
        result = fn(num1, num2)
        return result

    return inner
```

```
# 被带有参数的装饰器装饰的函数
@decorator('+')
def add(a, b):
    result = a + b
    return result


# 执行函数
result = add(1, 3)
print(result)
```

装饰器只能接收一个参数

带有参数的装饰器

需求

定义一个**既能装饰减法运算，又能装饰加法运算**的装饰器，即带有参数的装饰器。

```
#1.定义装饰器
def logging(flag):  # flag = "+"
    # 1.1 外部函数
    def decorator(fn):
        # 1.2内部函数
        def inner(num1, num2):
            # 判断流程
            if flag == "+":
                print("--正在努力加法计算--")
            elif flag == "-":
                print("--正在努力减法计算--")
            result = fn(num1, num2)
            return result
        # 返回内部函数
        return inner
    # 返回装饰器
    return decorator
```

```
# 2.被带有参数的装饰器装饰的函数
@logging('+')
def add(a, b):
    result = a + b
    return result

@logging('-')
def reduce(a, b):
    result = a - b
    return result


# 3.执行函数
result = add(1, 3)
print(result)
```

使用带有参数的装饰器，其实是在装饰器**外面又包裹**了一个函数，使用该函数接收参数，**返回装饰器**。

总结

1. 编写多个装饰器的代码过程是？

   离函数最近的装饰器先装饰，然后外面的装饰器再进行装饰，

   由内到外的装饰过程

2. 编写带有参数的装饰器代码的的过程是？

   使用带有参数的装饰器，其实是在装饰器外面又包裹了一个函数，

   使用该函数接收参数，返回装饰器。

目录

Contents

- 闭包介绍
- 闭包的使用
- 装饰器
- 装饰器的使用
- 多个装饰器的使用
- 带有参数装饰器
- 深浅拷贝

浅拷贝、深拷贝

1、可变类型与不可变类型

不可变对象：一旦创建就不可修改的对象，包括字符串、元组、数值类型（整型、浮点型）

该对象所指向的内存中的值不能被改变。当改变某个变量时候，由于其所指的值不能被改变，相当于把原来的值复制一份后再改变，这会开辟一个新的地址，变量再指向这个新的地址。

可变对象：可以修改的对象，包括列表、字典、集合

该对象所指向的内存中的值可以被改变。变量（准确的说是引用）改变后，实际上是其所指的值直接发生改变，并没有发生复制行为，也没有开辟新的地址，通俗点说就是原地改变。

黑马程序员
www.itheima.com

2、变量赋值执行原理

```
a = "python"
```

Python解释器干的事情：

① 创建变量a

② 创建一个对象(分配一块内存)，来存储值 'python'

③ 将变量与对象，通过指针连接起来，从变量到对象的连接称之为引用(变量引用对象)

黑马程序员
www.itheima.com

3、浅拷贝概念与原始数据准备

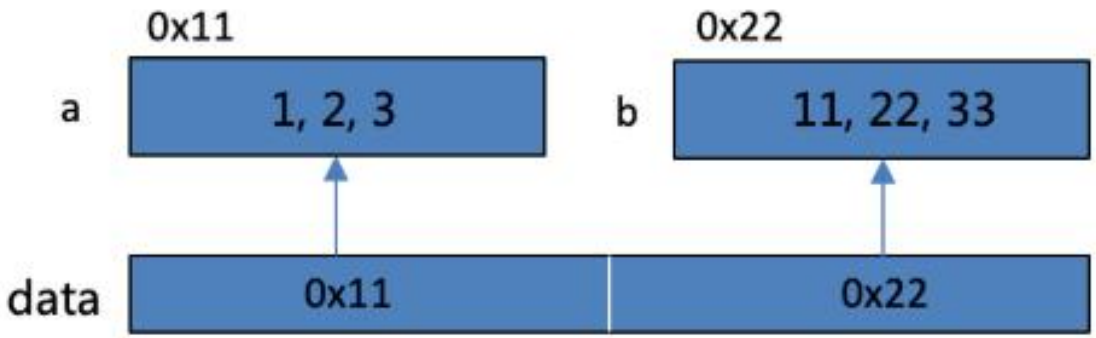
浅拷贝：创建新对象，其内容是原对象的引用。

浅拷贝之所以称为浅拷贝，是它仅仅只拷贝了一层，拷贝了最外围的对象本身，内部的元素都只是拷贝了一个引用而已

原始数据：

可变类型
a = [1, 2, 3]
b = [11, 22, 33]

有子对象的可变类型
data = [a, b]

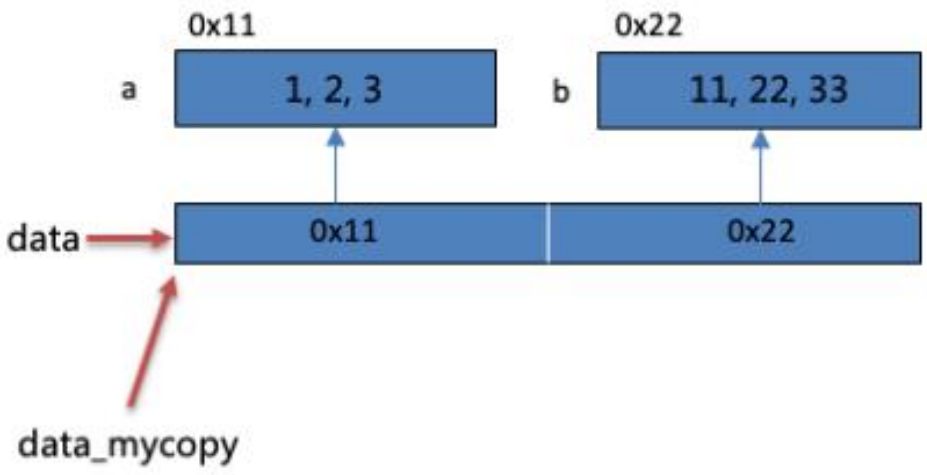


黑马程序员
www.itheima.com

4、可变类型浅拷贝

普通赋值

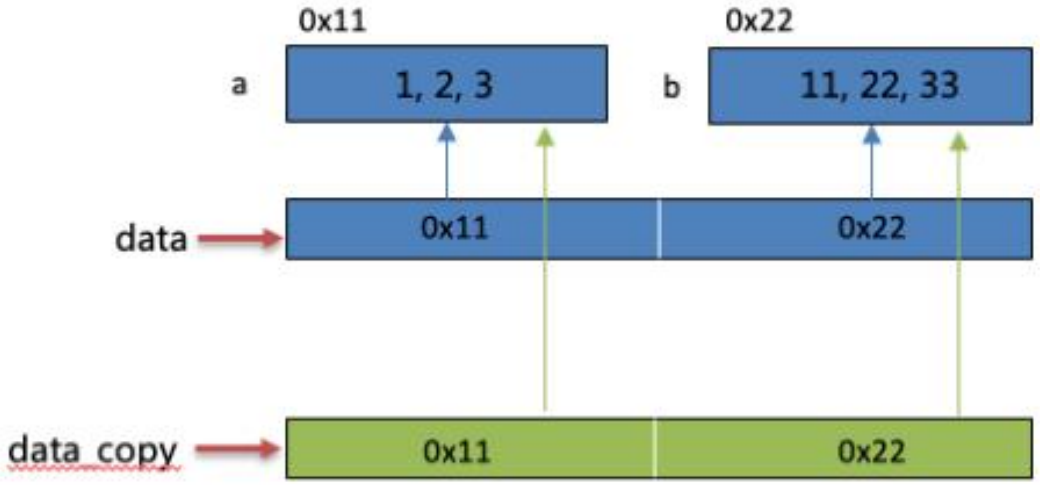
data_mycopy = data



data和data_mycopy指向**同一空间**

浅拷贝

data_copy = copy.copy(data)



data和data_copy指向**不同空间**

黑马程序员
www.itheima.com

4、不可变类型浅拷贝

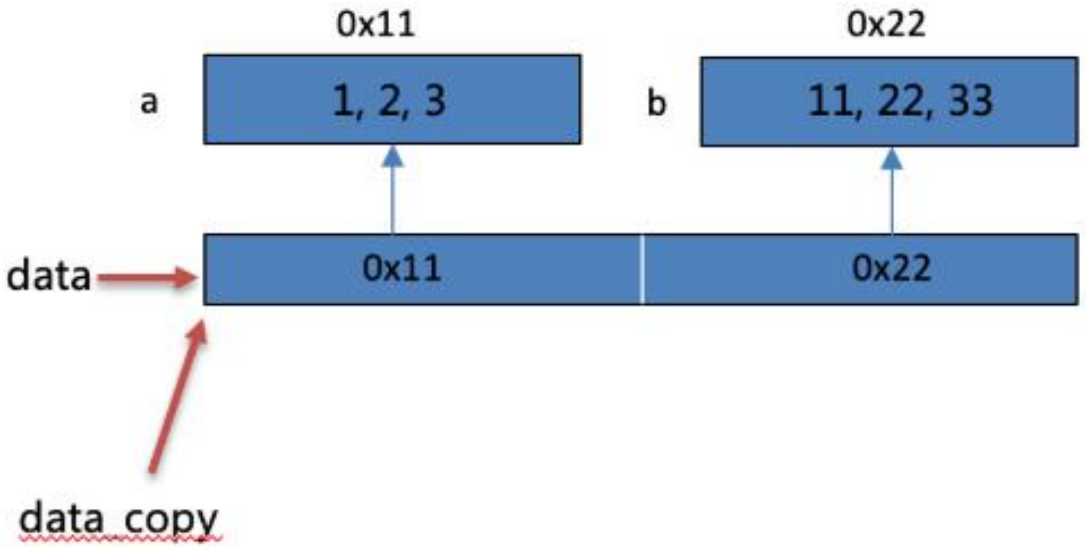
```
a = (1, 2, 3)
b = (11, 22, 33)
```

不可变类型

```
data = (a ,  b)
```

浅拷贝

```
data_copy = copy.copy(data)
```



黑马程序员
www.itheima.com

5、浅拷贝总结

1）当浅复制的值是不可变对象（字符串、元组、数值类型）时和“赋值”的情况一样，对象的id值（id()函数用于获取对象的内存地址）与浅复制原来的值相同。

2）当浅复制的值是可变对象（列表、字典、集合）时会产生一个“不是那么独立的对象”存在。

有两种情况：

第一种情况：复制的对象中无复杂子对象，原来值的改变并不会影响浅复制的值，同时浅复制的值改变也并不会影响原来的值。原来值的id值与浅复制原来的值不同。

第二种情况：复制的对象中有复杂子对象（例如列表中的一个子元素是一个列表），如果不改变其中复杂子对象，浅复制的值改变并不会影响原来的值。 但是改变原来的值中的复杂子对象的值会影响浅复制的值。

黑马程序员
www.itheima.com

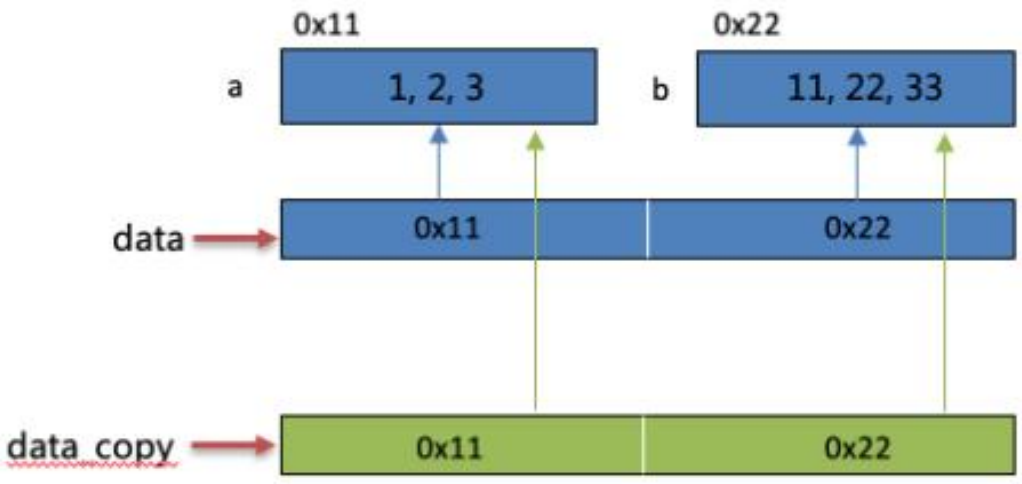
6、深拷贝

深拷贝：和浅拷贝对应，深拷贝拷贝了对象的所有元素，包括多层嵌套的元素。深拷贝出来的对象是一个全新的对象，不再与原来的对象有任何关联。所以改变原有被复制对象不会对已经复制出来的新对象产生影响。只有一种形式，copy模块中的deepcopy函数。

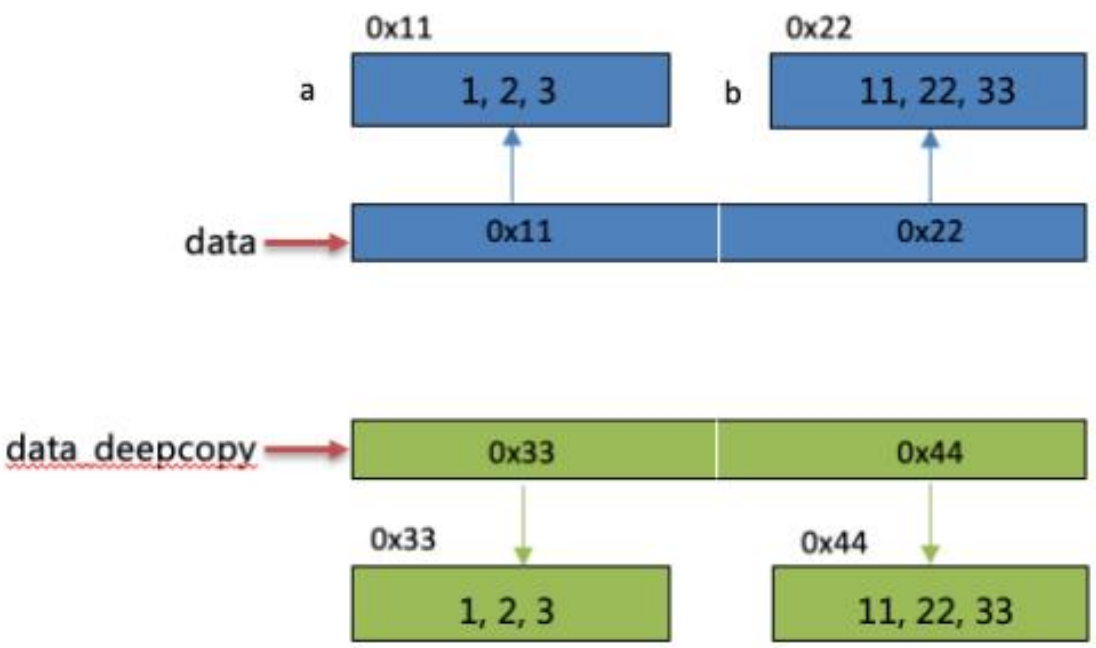
黑马程序员
www.itheima.com

7、可变类型深拷贝

浅拷贝
data_copy = copy.copy(data)



深拷贝
data_deepcopy = copy.deepcopy(data)



在拷贝可变类型对象的时候:
①浅拷贝只拷贝第一层中的数据并开辟空间储存
②深拷贝拷贝所有的数据并开辟对应的空间存储

黑马程序员
www.itheima.com

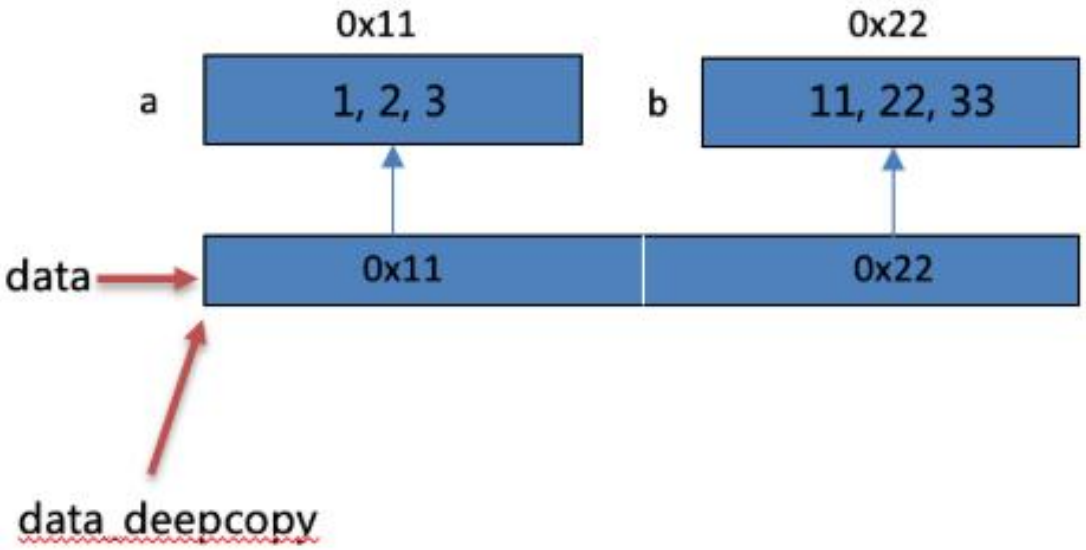
8、不可变类型深拷贝

不可变类型深拷贝：不可变类型进行深拷贝不会给拷贝的对象开辟新的内存空间，而只是拷贝了这个对象的引用。

```
a = (1, 2, 3)
b = (11, 22, 33)

不可变类型
data = (a ,  b)

深拷贝
data_deepcopy = copy.deepcopy(data)
```



总结

1. 什么是浅拷贝？

答：仅拷贝对象的引用，新的对象与原对象共享内存中的子对象。如果子对象发生变化，两个对象都会受影响。

2. 什么是深拷贝？

答：拷贝对象及其所包含的所有子对象的独立副本。新对象与原对象完全独立，修改新对象不会影响原对象。

黑马程序员
www.itheima.com
传智教育旗下高端IT教育品牌